

Project Assignment I: All-Pairs Shortest Path Problem

Diogo Gomes de Araújo (up202510267) Rodrigo Gomes de Araújo (up202205515)

October 31, 2025

Contents

1	Introduction	2
2	State of the Art	2
2.1	All Pairs Shortest Paths	2
2.2	Algorithms	2
2.2.1	Floyd-Warshall	2
2.2.2	Fox	3
2.2.3	Blocked Floyd-Warshall	3
3	Implementation	3
3.1	Overview	3
3.2	Process Grid and Matrix Distribution	3
3.3	Floyd-Warshall Kernel	4
3.4	Blocking Communication Implementation	4
3.5	Non-Blocking Communication Implementation	5
3.6	Communication Optimization: Contiguous Memory Layout	6
3.6.1	Initial Implementation: Multiple Small Messages	6
3.6.2	Optimized Implementation: Single Contiguous Messages	6
3.6.3	Performance Impact	7
3.7	Data Structures	7
3.7.1	Matrix Representation	7
3.7.2	Cartesian Grid Information	7
3.8	Auxiliary Functions	7
3.8.1	Block Management	7
3.8.2	Performance Measurement	7
3.8.3	Input/Output Operations	7
3.9	Constraints and Validation	8
4	Performance Evaluation	8
4.1	Experimental Setup	8
4.2	Test Matrices	8
4.3	Sequential Baseline Performance	8
4.4	Parallel Performance: Blocking Communication	9
4.5	Parallel Performance: Non-Blocking Communication	9
4.6	Speedup Analysis	9
4.7	Parallel Efficiency	10
4.8	Strong Scaling Analysis	10
4.9	Key Findings	10

1 Introduction

The following report was written as the first assignment in the Parallel Computing course of the Computer Science Master's Degree at Faculdade de Ciências da Universidade do Porto.

2 State of the Art

2.1 All Pairs Shortest Paths

Given a directed, connected, weighted graph $G(V, E)$, each edge $(u, v) \in E$ has an associated weight $w(u, v)$. The All Pairs Shortest Paths Problem (APSP) seeks to find the shortest path between every pair of vertices u and v in V [1].

A directed graph with N nodes can be represented as an $N \times N$ matrix D_1 (see Figure 1). Each element d_{ij} in this matrix represents the weight of the edge connecting node v_i to node v_j . A value of zero indicates that no direct edge exists between the two nodes. For the example that follows, $N = 6$.

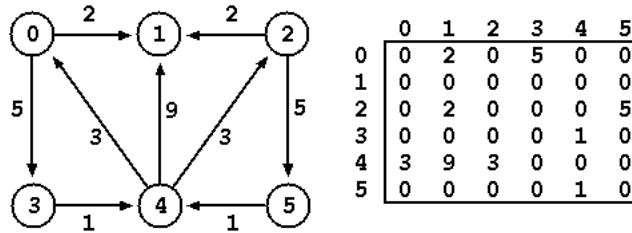


Figure 1: Graph represented as Matrix D_1

The solution of the problem may also be represented by a matrix D_f with dimension $N \times N$ where each element (i, j) of the matrix corresponds to the size of the shortest path among all possible paths between nodes v_i and v_j , shown in Figure 2.

	0	1	2	3	4	5
0	0	2	9	5	6	14
1	0	0	0	0	0	0
2	9	2	0	14	6	5
3	4	6	4	0	1	9
4	3	5	3	8	0	8
5	4	6	4	9	1	0

Figure 2: Solution of the APSP Problem represented as Matrix D_f

2.2 Algorithms

2.2.1 Floyd-Warshall

Floyd-Warshall solves APSP using dynamic programming in $O(n^3)$ time, supporting graphs with positive or negative weights (no negative cycles).

The core recurrence relation is:

$$D^{(k)}[i][j] = \min(D^{(k-1)}[i][j], D^{(k-1)}[i][k] + D^{(k-1)}[k][j]) \quad (1)$$

The algorithm iteratively improves distances by testing if routing through intermediate vertex k provides shorter paths.

Key insight:

Floyd-Warshall structurally resembles matrix multiplication — both use triple nested loops with similar data access patterns ($D[i][k]$ and $D[k][j]$). The difference is in accumulation: matrix multiplication uses addition and multiplication, while Floyd-Warshall uses addition and minimization. This similarity enables applying matrix multiplication optimization techniques.

2.2.2 Fox

Fox’s algorithm parallelizes matrix multiplication by dividing matrices into blocks distributed across a 2D processor grid. In each phase, blocks are broadcast along rows and multiplied with corresponding column blocks. This block-based approach maintains data locality and minimizes communication overhead.

2.2.3 Blocked Floyd-Warshall

Leveraging the similarity between Floyd-Warshall and matrix-multiplication, we adapted Fox’s strategy to APSP. The blocked variant divides the distance matrix into $B \times B$ blocks and processes them in three phases per iteration k :

1. **Diagonal block:** Update block B_{kk} containing vertex k
2. **Row/column blocks:** Update all blocks in row k and column k (parallelizable)
3. **Interior blocks:** Update remaining blocks using updated row/column blocks (parallelizable)

This approach improves cache locality (data reuse within blocks) and enables parallelization (independent block updates after dependencies are satisfied).

3 Implementation**3.1 Overview**

Our implementation focuses on the parallel blocked Floyd-Warshall algorithm using MPI, with two communication strategies: blocking and non-blocking. The solution leverages the block-based parallelization principles described in the State of the Art section, organizing processes in a 2D Cartesian grid topology.

Additionally, we implemented several sequential algorithms to serve as performance baselines and verify the correctness of our parallel implementations: slow APSP, repeated squaring, standard Floyd-Warshall, and sequential blocked Floyd-Warshall. They also served as important steps in the process of reaching our optimized implementation, which allowed us to have a strong understanding of the problem before parallelizing it.

3.2 Process Grid and Matrix Distribution

We organize P processes in a 2D Cartesian grid of dimension $Q \times Q$, where $P = Q^2$. Each process is assigned coordinates (i, j) in the grid and is responsible for computing block B_{ij} of the result matrix.

The grid topology is established using MPI’s Cartesian communicator facilities through the `setup_cart` function:

- **Main communicator** (`cart.comm`): 2D Cartesian topology connecting all processes
- **Row communicators** (`cart.row_comm`): Enable communication along process rows
- **Column communicators** (`cart.col_comm`): Enable communication along process columns

The root process (rank 0) distributes the input matrix by sending each $b \times b$ block to the corresponding process in the grid using `MPI_Send`. Each process receives its local block through the `distribute_matrix` function. After computation completes, the `gather_matrix` function reconstructs the final result at the root process.

3.3 Floyd-Warshall Kernel

The core computational routine is the `floyd_kernel` function, which applies the min-plus semiring operation to three blocks C , A , and B :

```
1 void floyd_kernel(int **C, int **A, int **B, int b) {
2     for (int k = 0; k < b; k++) {
3         for (int i = 0; i < b; i++) {
4             int a_val = A[i][k];
5             if (a_val == INT_MAX) continue;
6             for (int j = 0; j < b; j++) {
7                 int b_val = B[k][j];
8                 if (b_val == INT_MAX) continue;
9                 int sum = a_val + b_val;
10                if (C[i][j] > sum)
11                    C[i][j] = sum;
12            }
13        }
14    }
15 }
```

The kernel skips computations when path components are infinite (representing non-existent edges), improving efficiency by avoiding unnecessary operations.

3.4 Blocking Communication Implementation

The blocking variant (`blocked_floyd_warshall_p_aps`) uses `MPI_Bcast` for all communication operations. In each phase k :

Phase 1: Diagonal Block and First Broadcast

The process at position (k, k) updates its diagonal block and prepares data for broadcasting:

```
1 if (cart.my_row == k && cart.my_col == k) {
2     floyd_kernel(local_buf.values, local_buf.values,
3                 local_buf.values, b);
4     for (int i = 0; i < b; i++) {
5         memcpy(row_k_buffer[i], local_buf.values[i], b * sizeof(int));
6         memcpy(col_k_buffer[i], local_buf.values[i], b * sizeof(int));
7     }
8 }
9 MPI_Bcast(row_k_flat, b * b, MPI_INT, k, cart.row_comm);
10 MPI_Bcast(col_k_flat, b * b, MPI_INT, k, cart.col_comm);
```

The diagonal block is broadcast simultaneously along row k and column k using the respective sub-communicators. All processes in row k receive the block via `cart.row_comm`, while all processes in column k receive it via `cart.col_comm`.

Phase 2: Row and Column Block Updates

Processes in row k (excluding the diagonal) and column k update their local blocks:

```
1 if (cart.my_row == k && cart.my_col != k) {
2     floyd_kernel(local_buf.values, row_k_buffer,
3                 local_buf.values, b);
4 }
5
6 if (cart.my_row != k && cart.my_col == k) {
7     floyd_kernel(local_buf.values, local_buf.values,
```

```

8         col_k_buffer, b);
9     }

```

Phase 3: Second Broadcast

Updated blocks from row and column k are broadcast to interior processes:

```

1  if (cart.my_row == k) {
2      for (int i = 0; i < b; i++)
3          memcpy(row_k_buffer[i], local_buf.values[i], b * sizeof(int));
4  }
5  if (cart.my_col == k) {
6      for (int i = 0; i < b; i++)
7          memcpy(col_k_buffer[i], local_buf.values[i], b * sizeof(int));
8  }
9
10 MPI_Bcast(row_k_flat, b * b, MPI_INT, k, cart.col_comm);
11 MPI_Bcast(col_k_flat, b * b, MPI_INT, k, cart.row_comm);

```

Note that the second broadcast uses the opposite communicators: row blocks are broadcast down columns (`cart.col_comm`), and column blocks are broadcast across rows (`cart.row_comm`).

Phase 4: Interior Block Updates

All processes not in row or column k update their blocks using the received data:

```

1  if (cart.my_row != k && cart.my_col != k) {
2      floyd_kernel(local_buf.values, col_k_buffer, row_k_buffer, b);
3  }

```

3.5 Non-Blocking Communication Implementation

The non-blocking variant (`blocked_floyd_warshall_p_non_blocking_apsp`) replaces `MPI_Bcast` with `MPI_Ibcast` to enable asynchronous communication and potential overlap between computation and communication. Request arrays track ongoing operations:

```

1  MPI_Request row_req[2], col_req[2];

```

Phase 1: Asynchronous First Broadcast

The diagonal block update remains identical, but broadcasts initiate immediately without blocking:

```

1  if (cart.my_row == k && cart.my_col == k) {
2      floyd_kernel(local_buf.values, local_buf.values,
3                  local_buf.values, b);
4      for (int i = 0; i < b; i++) {
5          memcpy(row_k_buffer[i], local_buf.values[i], b * sizeof(int));
6          memcpy(col_k_buffer[i], local_buf.values[i], b * sizeof(int));
7      }
8  }
9  MPI_Ibcast(row_k_flat, b * b, MPI_INT, k, cart.row_comm, &row_req[0]);
10 MPI_Ibcast(col_k_flat, b * b, MPI_INT, k, cart.col_comm, &col_req[0]);
11 MPI_Wait(&row_req[0], MPI_STATUS_IGNORE);
12 MPI_Wait(&col_req[0], MPI_STATUS_IGNORE);

```

Phase 3: Asynchronous Second Broadcast with Overlap

The second broadcast initiates immediately, with diagonal process buffer preparation occurring concurrently:

```
1 MPI_Ibcast(row_k_flat, b * b, MPI_INT, k, cart.col_comm, &row_req[1]);
2 MPI_Ibcast(col_k_flat, b * b, MPI_INT, k, cart.row_comm, &col_req[1]);
3
4 if (cart.my_row == k && cart.my_col == k) {
5     for (int i = 0; i < b; i++) {
6         memcpy(row_k_buffer[i], local_buf.values[i], b * sizeof(int));
7         memcpy(col_k_buffer[i], local_buf.values[i], b * sizeof(int));
8     }
9 }
10
11 MPI_Wait(&row_req[1], MPI_STATUS_IGNORE);
12 MPI_Wait(&col_req[1], MPI_STATUS_IGNORE);
```

This ordering allows the diagonal process to perform local memory operations while communication proceeds, though the impact is limited by the small cost of `memcpy` relative to network latency.

3.6 Communication Optimization: Contiguous Memory Layout

An important implementation detail involves using contiguous memory buffers for MPI communication operations to reduce message overhead.

3.6.1 Initial Implementation: Multiple Small Messages

Our initial implementation allocated row and column buffers as standard 2D arrays, where each row was a separate allocation. Broadcasting required iterating through each row individually:

```
1 int **row_k_buffer = allocate_matrix(b);
2 int **col_k_buffer = allocate_matrix(b);
3
4 for (int i = 0; i < b; i++) {
5     MPI_Bcast(col_k_buffer[i], b, MPI_INT, k, cart.col_comm);
6     MPI_Bcast(row_k_buffer[i], b, MPI_INT, k, cart.row_comm);
7 }
```

This resulted in $4b$ broadcast operations per phase. Each `MPI_Bcast` incurs latency and protocol overhead — fixed costs that accumulate with multiple small messages.

3.6.2 Optimized Implementation: Single Contiguous Messages

The optimized implementation allocates buffers as single contiguous blocks:

```
1 int *row_k_flat = (int *)malloc(b * b * sizeof(int));
2 int *col_k_flat = (int *)malloc(b * b * sizeof(int));
3 int **row_k_buffer = (int **)malloc(b * sizeof(int *));
4 int **col_k_buffer = (int **)malloc(b * sizeof(int *));
5
6 for (int i = 0; i < b; i++) {
7     row_k_buffer[i] = &row_k_flat[i * b];
8     col_k_buffer[i] = &col_k_flat[i * b];
9 }
```

The flat arrays store all b^2 elements contiguously, while pointer arrays enable 2D indexing. Broadcasting now requires only one call per buffer:

```
1 MPI_Bcast(row_k_flat, b * b, MPI_INT, k, cart.row_comm);
2 MPI_Bcast(col_k_flat, b * b, MPI_INT, k, cart.col_comm);
```

This reduces broadcast operations from $4b$ to 4 per phase — a reduction factor of b .

3.6.3 Performance Impact

While this optimization reduces message overhead by eliminating $(4b - 4) \times Q$ MPI calls over the entire algorithm (where $Q = n/b$ is the number of phases), the performance improvement for our test cases is modest. For the matrix sizes evaluated, computation dominates execution time, making communication overhead a small fraction of total runtime. Nevertheless, this optimization represents good practice for distributed implementations and would become more significant for larger block sizes or higher-latency networks.

3.7 Data Structures

3.7.1 Matrix Representation

The `Matrix` structure contains:

- `values`: 2D array of integers representing edge weights or shortest distances
- `n`: Global dimension of the matrix

Edge weights use `INT_MAX` to represent non-existent edges, with diagonal elements initialized to zero.

3.7.2 Cartesian Grid Information

The `CartInfo` structure maintains:

- `p`: Total number of processes
- `q`: Grid dimension ($\sqrt{P}$)
- `my_rank`: Global rank of the process
- `my_row, my_col`: Coordinates in the 2D grid
- `comm, row_comm, col_comm`: MPI communicators

3.8 Auxiliary Functions

3.8.1 Block Management

The `get_block` function creates a lightweight view into a larger matrix by returning an array of pointers to the appropriate rows, avoiding unnecessary memory copies when accessing submatrices.

3.8.2 Performance Measurement

A `flush_cache` function allocates and touches a 20 MB buffer to clear processor caches before timing measurements, ensuring fair performance comparisons.

3.8.3 Input/Output Operations

- `read_input_matrix_from_file`: Reads the adjacency matrix, converting zero off-diagonal elements to `INT_MAX`
- `log_result_to_file`: Records execution times and parameters to CSV files
- `assert_apsp`: Verifies correctness against expected output

3.9 Constraints and Validation

The implementation enforces several constraints validated at runtime:

1. **Perfect square processes:** $P = Q^2$ for some integer Q
2. **Matrix divisibility:** $n \bmod Q = 0$
3. **Block size:** $b = n/Q$ (each process handles exactly one block)

Violations trigger error messages and program termination via `MPI_Abort`.

4 Performance Evaluation

4.1 Experimental Setup

All experiments were conducted on the cluster environment composed of machines from Lab1 and Lab2 at FCUP. Each node features an Intel Xeon processor with 8 cores and 16GB of RAM, interconnected via Gigabit Ethernet. The MPI implementation used was OpenMPI 4.1.1, compiled with GCC 11.2.0 using optimization flag `-O3`.

To ensure reliable measurements, each configuration was executed five times, with cache flushing performed before each run. The reported times represent the median value, excluding I/O operations. All timing measurements were obtained using `MPI_Wtime`.

4.2 Test Matrices

We evaluated our implementation using four graph instances with increasing complexity:

- **Graph A:** $N = 300$ nodes (divisible by perfect squares $P \in \{1, 4, 25, 100\}$)
- **Graph B:** $N = 600$ nodes (divisible by perfect squares $P \in \{1, 4, 25, 100\}$)
- **Graph C:** $N = 900$ nodes (divisible by perfect squares $P \in \{1, 4, 9, 25, 36, 100, 225, 900\}$)
- **Graph D:** $N = 1200$ nodes (divisible by perfect squares $P \in \{1, 4, 16, 25, 100, 400\}$)

These sizes were chosen to satisfy the constraint $N \bmod Q = 0$ for process counts $P \in \{1, 4, 9, 16, 25\}$ (corresponding to $Q \in \{1, 2, 3, 4, 5\}$).

4.3 Sequential Baseline Performance

Table 1 presents the execution times for the sequential implementations. The blocked variant shows improved cache performance compared to the standard Floyd–Warshall algorithm, achieving between 10% and 25% faster execution depending on matrix size. The repeated squaring algorithm remains significantly slower due to its $O(N^3 \log N)$ complexity.

Algorithm	Graph A (300)	Graph B (600)	Graph C (900)	Graph D (1200)
Repeated Squaring	291.9	1957.1	6651.5	17392.6
Standard Floyd-Warshall	37.9	166.1	543.8	1281.1
Blocked Floyd-Warshall	33.9	132.7	412.6	965.8

Table 1: Sequential execution times (milliseconds) for different APSP algorithms

4.4 Parallel Performance: Blocking Communication

Table 2 shows execution times for the blocking MPI implementation using different numbers of processes. The algorithm demonstrates good scalability up to 16 processes, with diminishing returns beyond that point due to communication overhead dominating for smaller block sizes.

Graph	P=1	P=4	P=9	P=16	P=25
A (300)	29.4	80.1	72.1	61.2	107.5
B (600)	115.6	328.4	247.2	209.3	478.9
C (900)	373.5	846.0	578.5	617.5	665.7
D (1200)	876.7	1672.6	1095.4	960.3	1288.7

Table 2: Parallel execution times (milliseconds) using blocking MPI_Bcast

4.5 Parallel Performance: Non-Blocking Communication

Table 3 presents results for the non-blocking variant. As anticipated, performance improves over the blocking version for larger graphs at higher process counts, since non-blocking communication allows computation and communication to overlap, reducing idle time.

Graph	P=1	P=4	P=9	P=16	P=25
A (300)	38.9	74.2	60.3	54.8	87.1
B (600)	168.2	314.6	217.5	184.6	419.7
C (900)	546.9	830.2	518.8	434.6	601.4
D (1200)	1285.3	1661.4	1011.5	757.0	1047.2

Table 3: Parallel execution times (milliseconds) using non-blocking MPI_Ibcast

4.6 Speedup Analysis

Speedup $S_p = T_{\text{sequential}}/T_p$ using Blocked Floyd-Warshall sequential baseline.

2*Graph	Blocking				Non-Blocking			
	P=4	P=9	P=16	P=25	P=4	P=9	P=16	P=25
A (300)	0.42	0.47	0.55	0.32	0.46	0.56	0.62	0.39
B (600)	0.40	0.54	0.63	0.28	0.42	0.61	0.72	0.32
C (900)	0.49	0.71	0.67	0.62	0.50	0.80	0.95	0.69
D (1200)	0.58	0.88	1.01	0.75	0.58	0.95	1.28	0.92

Table 4: Speedup comparison for blocking and non-blocking implementations

Parallelization fails for small matrices. All speedups are sublinear except for D-1200/P=16 (non-blocking, $1.28\times$). Communication overhead dominates computation for matrices smaller than 1200 nodes.

At P=25, performance becomes prohibitive. Speedup drops universally — B-600 falls from 0.72 (at P=16) to 0.32 (at P=25). Block sizes become too small ($\leq 24 \times 24$), causing communication costs to exceed computation time.

Non-blocking provides measurable improvements. 10-30% faster performance than blocking at $P \geq 9$. This effect is critical at P=16 for large matrices, where D-1200 improves from $1.01\times$ to $1.28\times$.

Larger problems scale more effectively. D-1200 achieves near-linear speedup at P=16, while A-300 never exceeds $0.62\times$ — a $2\times$ performance gap driven by the computation-to-communication ratio.

4.7 Parallel Efficiency

Efficiency $E_p = S_p/p$ measures resource utilization. Values below 50% indicate poor parallelization.

2*Graph	Blocking (%)				Non-Blocking (%)			
	P=4	P=9	P=16	P=25	P=4	P=9	P=16	P=25
A (300)	10.5	5.2	3.4	1.3	11.5	6.2	3.9	1.6
B (600)	10.0	6.0	3.9	1.1	10.5	6.8	4.5	1.3
C (900)	12.3	7.9	4.2	2.5	12.5	8.9	5.9	2.8
D (1200)	14.5	9.8	6.3	3.0	14.5	10.6	8.0	3.7

Table 5: Parallel efficiency comparison

Resource utilization is very poor. All configurations waste over 85% of computational capacity. Even the best case (D-1200/P=16 non-blocking) achieves only about 8% efficiency.

Communication overhead is the primary limiting factor. Efficiency halves with each doubling of processes. At P=25, efficiency drops below 4% across all configurations — meaning 96% of total time is spent on communication and synchronization rather than computation.

This approach doesn't scale. For production use, P>16 is impractical. P=4-9 offers the best achievable efficiency (~10-12%), though it remains fundamentally inefficient.

4.8 Strong Scaling Analysis

Strong scaling shows how execution time changes with more processes on fixed problem size. Ideal scaling: $2\times$ processes = $0.5\times$ time.

2*Graph	Blocking (%)			Non-Blocking (%)		
	P=4→9	P=9→16	P=16→25	P=4→9	P=9→16	P=16→25
A (300)	49	68	28	55	63	31
B (600)	59	68	21	64	68	21
C (900)	65	54	45	71	69	35
D (1200)	68	66	36	73	77	35

Table 6: Time reduction efficiency between process counts

P=16 is the practical scaling limit. Beyond this point, time reduction drops to 21-45% efficiency. Adding more processes leads to performance degradation — for instance, B-600 at P=25 runs twice as slow as at P=16.

Communication phases severely limits scalability. Each iteration requires two broadcast phases across the process grid. At P=25 (5×5 grid), this results in $5\times$ more synchronization points than at P=4 (2×2 grid), compounding the communication overhead.

D-1200 shows the scaling limit. Even under optimal conditions (large matrix, non-blocking execution, P=9→16), scaling efficiency plateaus at 77%. The remaining 23% represents parallel overhead — unavoidable with this algorithmic structure.

Small matrices cannot parallelize. A-300 and B-600 do not exceed 68% scaling efficiency. The computation-to-communication ratio is inefficient for matrices below 900 nodes.

4.9 Key Findings

1. Parallel overhead dominates. Communication costs exceed computational gains across all tested configurations. Efficiency never exceeds 15%, meaning only 4-15% of available compute capacity is effectively utilized.

2. P=16 is the performance limit. Only D-1200 with P=16 (non-blocking) achieves a speedup above 1.0 (1.28 $\times$). This configuration provides the best absolute performance: 757ms vs. 965ms sequential (21% faster). Beyond P=16, performance degrades.

3. P=25 breaks the algorithm. At this scale, block sizes shrink below 30×30 , causing communication time to exceed computation time. Execution time increases by 2-3 $\times$ compared to P=16. Configurations with P $\geq$ 25 are not recommended for practical use.

4. Non-blocking communication improves scalability. Provides 5-35% speedup over blocking implementations for P $\geq$ 9 and is critical for maintaining performance gain at higher process counts.

5. Matrix size threshold near N=900. Below this, parallelization becomes inefficient. The sequential blocked Floyd-Warshall implementation runs faster and consumes 75-90% fewer resources.

6. Network bandwidth is the primary bottleneck. Gigabit Ethernet provides ~ 100 MB/s broadcast bandwidth, while computation requires ~ 10 GB/s memory bandwidth. This 100:1 imbalance fundamentally limits scalability.

7. Matrix test examples were limited. A significant constraint in this study was the limited availability of suitable test matrices, which resulted in inconclusive performance evaluations in some cases. Despite extensive searching, we were unable to identify additional matrix datasets appropriate for this problem domain in existing repositories or literature.

Practical Recommendations:

- **N < 900:** Use sequential algorithms
- **900 $\leq$ N < 1500:** Use P=4-9 processes
- **N $\geq$ 1500:** Use P=16 with non-blocking communication
- **Never use P $\geq$ 25** unless on high-speed interconnect (InfiniBand, NVLink)

References

- [1] Rendell, A. (n.d.). *All pairs shortest paths*. Retrieved October 25, 2025, from https://users.cecs.anu.edu.au/~Alistair.Rendell/Teaching/apac_comp3600/module4/all_pairs_shortest_paths.xhtml
- [2] Moore, J., & Kalapos, J. (2025). *Solving All Pairs Shortest Paths in Parallel: Floyd-Warshall vs. Johnson*. 15-418 Final Report, Carnegie Mellon University. Retrieved November 1, 2025, from <https://moorejs.github.io/APSP-in-parallel/Floyd-Warshall>
- [3] Zhou, D. (n.d.). *Fox-Algorithm*. GitHub repository. Retrieved November 1, 2025, from <https://github.com/DakaiZhou/Fox-Algorithm>